



A 3-LAYER NEURAL NETWORK

PRESENTED BY

- Adithye Nambiar
 - Soham Kumthekar
 - Mohammad Asad Akhtar
 - Tanay Joshi
 - Sif Al Masalmeh
- 



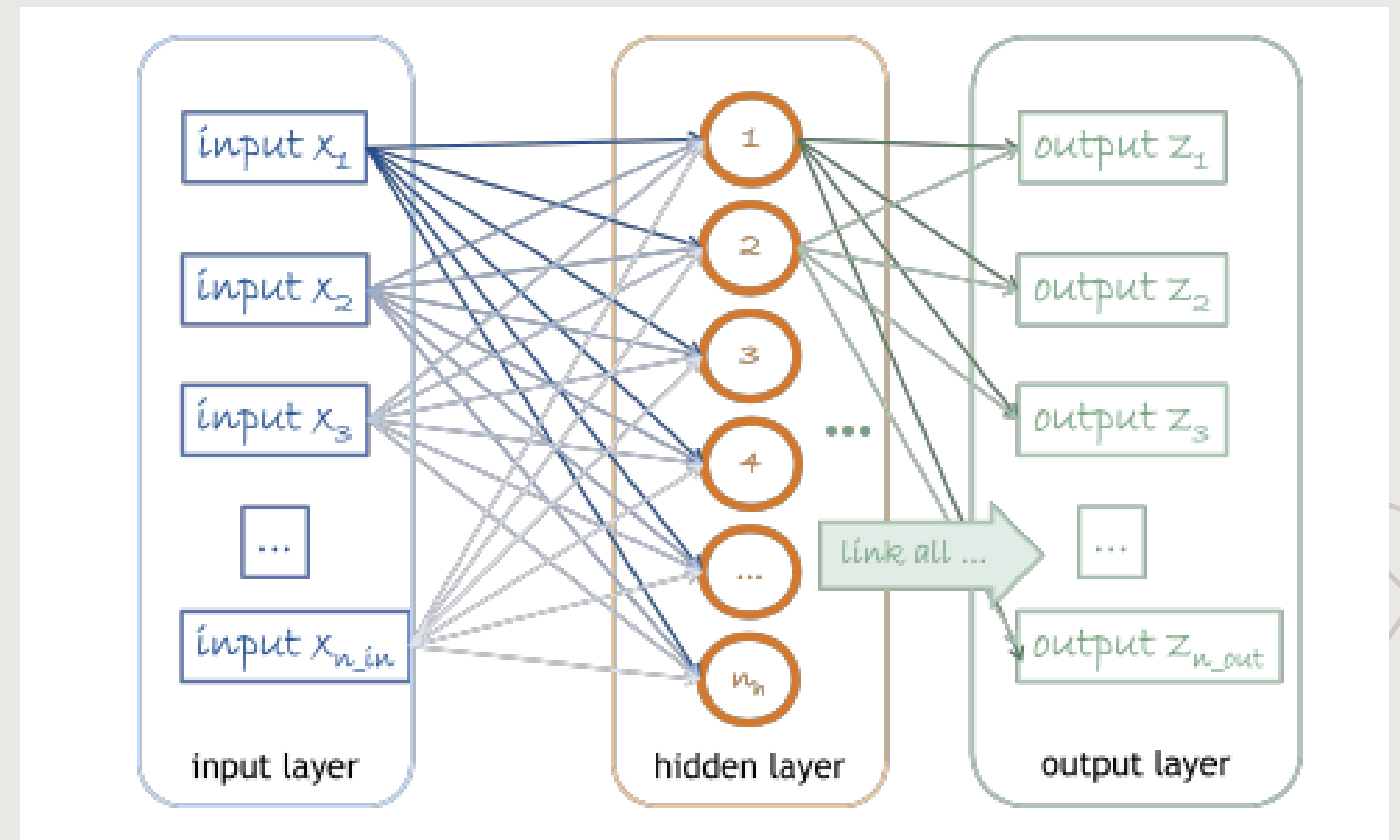
PROJECT OVERVIEW



What is a Neural Network?

Computational Models inspired by the brain

- Computational models inspired by biological neural networks
- Made up of interconnected **nodes** (neurons) arranged in **layers**
- Each neuron receives **inputs**, applies **weight** to inputs, and passes the result through an **activation function**
- A 3-layer feedforward network includes an **input** layer for features, a **hidden** layer for pattern extraction, and an **output** layer for predictions
- **Training** involves adjusting **weights** using **backpropagation** to minimize prediction errors

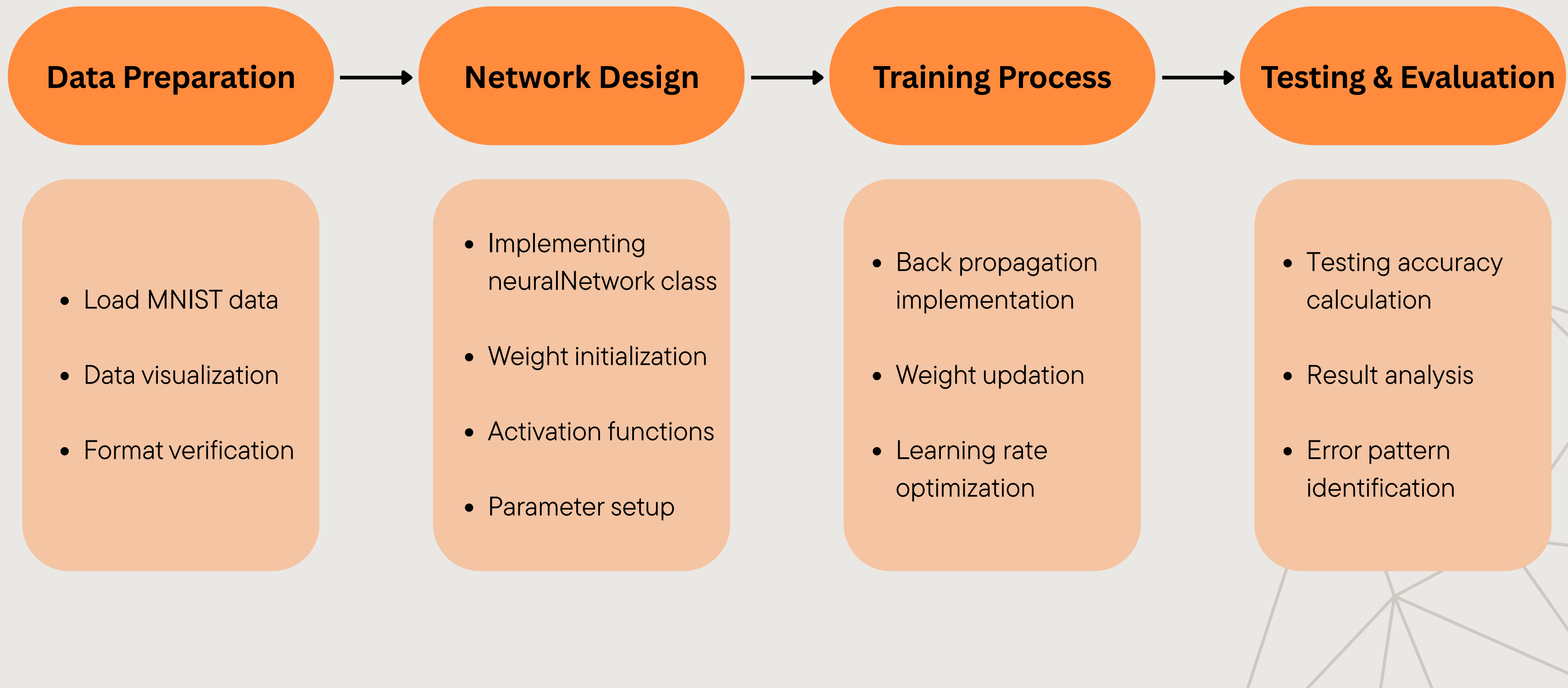


Tasks to do

- **Visualize MNIST data** to understand input distribution
- Implement **neuralNetwork class** in MATLAB
- **Train** using large **MNIST dataset**
- **Test** and **evaluate** performance
- Experiment with different **random initializations**



Neural Network Implementation





DATA PREPARATION

Plotting numbers (visualizing data)

```
clc;
clear;

filename = fullfile('datasets', 'mnist_test_10.csv');
data = readmatrix(filename);
index = randi(10);

% specify digit
label = data(index, 1);
pixels = data(index, 2:end);

image = reshape(pixels, [28, 28]);
figure;
imshow(image, []);
title(['Label: ', num2str(label)]);
```

Plotting single digits

```
clear; clc;
% Load the MNIST CSV file
filename= fullfile('datasets', 'mnist_train_100.csv');
data = readmatrix(filename);

hold on; %turn off for one digit one graph
digits_to_plot = 0:9;

for digit = digits_to_plot
    % Find indices of this digit in the dataset
    idx = find((data(:, 1) == digit));
    % Take first N samples
    samples_per_digit=length(idx);

    for j = 1:samples_per_digit
        sample = data(idx(j), 2:end);
        img = reshape(sample, [28, 28]);
        img = flipud(img); %flip

        % Calculate position for subplot-like grid
        x_offset = (j-1) * 28;
        y_offset = (-digit) * 28;

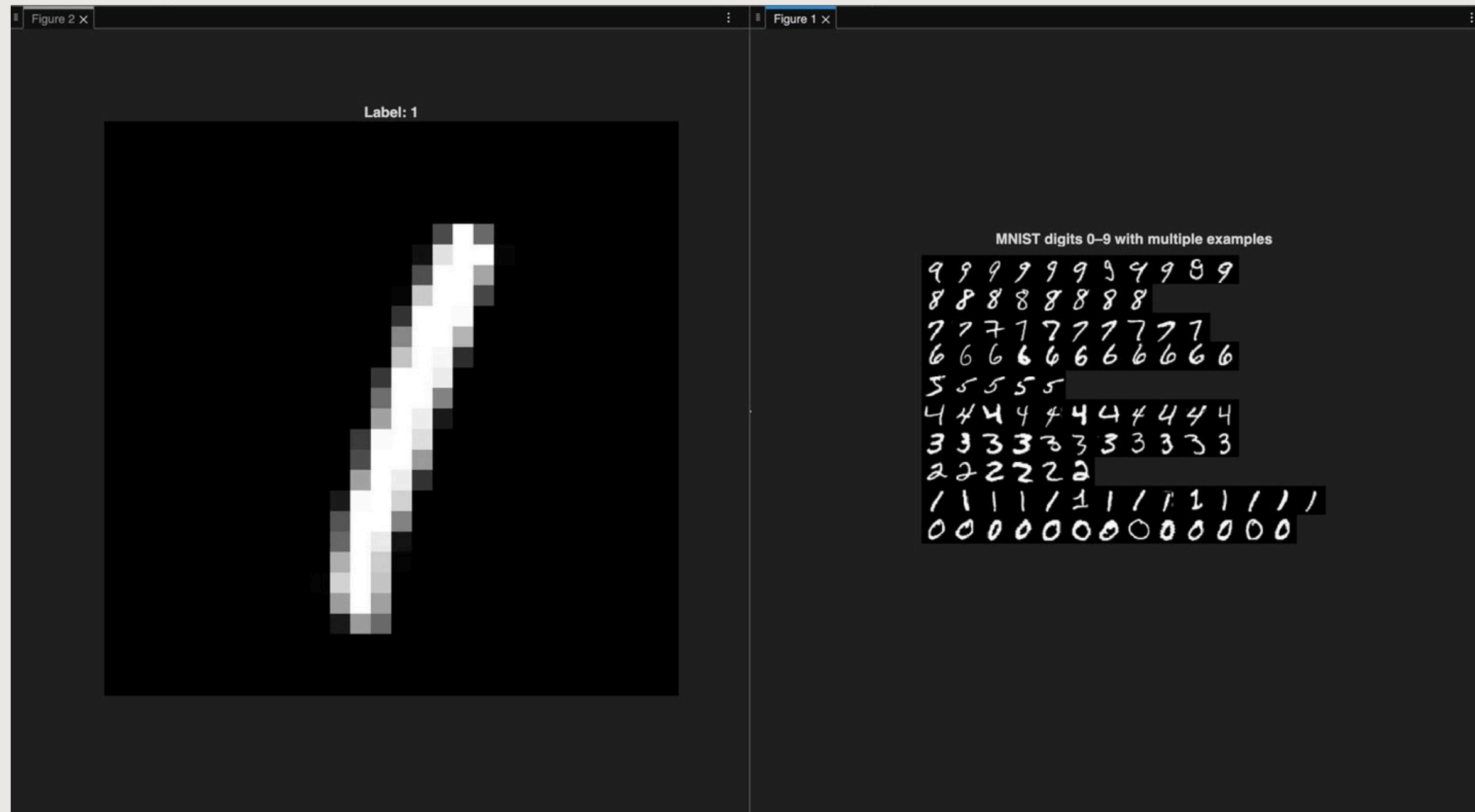
        % Display the image using imagesc with offset
        imagesc(x_offset + (1:28), y_offset + (1:28), img);
        colormap('gray');
    end
end

axis equal;

title('MNIST digits 0-9 with multiple examples');
```

Plotting multiple digits

Plotting numbers (visualizing data)



Output



NETWORK DESIGN

Neural Network Class

```
classdef neuralNetwork
    % neuralNetwork: Class for a three layer nN
    % A detailed explanation goes here ;- )
    properties
        inodes; % number of input nodes
        hnodes; % number of hidden nodes
        onodes; % number of output nodes
        lr;      % learning rate
        w_ih;    % link weights between input layer and hidden layer
        w_ho;    % link weights between hidden layer and output layer
    end

    methods
        function self = neuralNetwork(inputnodes, hiddennodes, outputnodes, learningrate)
            self.inodes=inputnodes;
            self.hnodes=hiddennodes;
            self.onodes=outputnodes;
            self.lr=learningrate;
            %Xavier initialization
            self.w_ih=randn(self.hnodes,self.inodes)/ sqrt(self.inodes);
            self.w_ho=randn(self.onodes,self.hnodes)/ sqrt(self.hnodes);

            %self.w_ih=zeros(self.hnodes,self.inodes) *0.5;
            %only guessing 0 when weights=0.5 and 1 when weights=0 (same guess with same weights)
            %self.w_ho=zeros(self.onodes,self.hnodes) *0.5;
        end

        % Query the neural network. The function takes a vector of inputs
        % and computes the outputs, i.e. a "prediction".
        function [final_outputs,hidden_output] = query(self, inputs)
            hidden_output=self.w_ih * (inputs);
            hidden_output = self.actfun(hidden_output);
            final_outputs = self.w_ho * hidden_output;
            final_outputs = self.actfun(final_outputs);
        end
    end
end
```



Neural Network Class

```
% Train the neural network. This function learns the link weights
% from the training data.
function self = train(self, inputs, targets)
    %error calculation
    [final,hidden]=self.query(inputs);
    output_error=targets-final;
    hidden_error=self.w_ho' * output_error;

    %training (changing weights)

    self.w_ho=self.w_ho + (self.lr*(output_error .* final .* (1-final))) * hidden';
    self.w_ih= self.w_ih + (self.lr*(hidden_error .* hidden .* (1-hidden))) * inputs';
end

%testing function
function correctness = test(self, inputs, target)
    [final,~]=self.query(inputs);
    %guess correctness
    [~,target_number]=max(target);
    [~,guessed_number]=max(final);
    correctness =(target_number==guessed_number);

end

end

methods(Static)
    function out = actfun(x)
        out=1./(1+exp(-x));
    end
end

end
```

neuralNetwork.m Part 2



Explaining the code

- The class **neuralNetwork** is defined with the properties: input nodes, hidden nodes, output nodes, learning rate, link weights between input and hidden layer and link weights between hidden and output layer.

Functions:

- The function **neuralNetwork(inputnodes, hiddennodes, outputnodes, learningrate)** is used as an **initialization** function. It initializes the neural network's structure and weights. The weight initialization is done through **Xavier Initialization**. In this initialization, weights are randomized and scaled by the inverse square root of the number of input connections. It helps prevent vanishing/exploding gradients during training.
- The **query** function performs a **forward pass** to compute the network's output (prediction) for a given input. It takes in the parameters of the **initialized neural network** and a vector of **inputs**. It returns two parameters: a vector of the **hidden layer** and a vector of the **output** and is an implementation of the function:

$$\vec{z} = nn(\vec{x}) = f_{act}(W_{hout} \cdot f_{act}(W_{inh} \cdot \vec{x}))$$

- The **train** function is responsible for **learning** the link weights from the training data using **backpropagation**. It first uses the **query** function to calculate the **hidden** and **output** layer activation. After that, the **error** at the **output** layer is calculated and then propagated back to the **hidden** layer using the **transpose** of the weights from hidden to output layer. The weights are then accordingly adjusted at the **input** using the **gradient descent rule**, which is:

$$\Delta W = \alpha \cdot [\vec{e}_k \cdot * \vec{s}_k \cdot * (1 - \vec{s}_k)] \cdot \vec{s}_j^T$$

Explaining the code (continued)

- The **test** function evaluates the network's **prediction accuracy** on a given input. It takes three parameters, namely the **initialized neural network**, a vector of **inputs** and a **target number**. The function first runs the **query** function and saves the returned vector of **outputs** into a variable named **final**. It then **returns 1** if the **target number** is **equal** to the **maximum** of the **output vector**, and **returns 0** otherwise.
- The **actfun** function is the **activation function** of the neural network; specifically the **sigmoid function**. It squashes input values to a range between 0 and 1 and is used in both hidden and output layers.

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$





TRAINING PROCESS

Training the Neural Network

```
clc;clear;

input_nodes=784;
hidden_nodes=100; %increasing hidden nodes increases accuracy and training time (and vice versa)
output_nodes=10;
learning_rate=0.2;

%intialize neural network
network1=neuralNetwork(input_nodes,hidden_nodes,output_nodes,learning_rate);
%load("Trained_models\trainedNetwork.mat"); %uncomment if training on same network
disp("Training in progress....");
tic;
% Load the MNIST CSV file
filename= fullfile('datasets\mnist_train.csv','mnist_train.csv');
data = readmatrix(filename);

%train
for i = 1:size(data,1)
    %preparing values using correction
    inputs = ((data(i,2:end) / 255.0)*0.99+0.01)';
    n=data(i,1);
    target = ones(10,1) ./ 100;
    target(n+1,1)=0.99;
    network1=network1.train(inputs, target);
end
fprintf("Time taken: %.1f s\n",toc);
disp("Saving in progress....")
%saving the trained model
save('Trained_models\trainedNetwork.mat', 'network1');

disp("done");
```

Training.m



Explaining the code

- The **training.m** file is used to **train** the neural network on the **MNIST handwritten digits** dataset using a CSV file (**mnist_train.csv**)
- The following parameters are given for initializing the neural network:
 1. Input nodes = 784
 2. Hidden nodes = 100
 3. Output nodes = 10
 4. Learning rate = 0.2
- The **inputs** are then **prepared** using the formula: $x_k = v_k/255.0 \cdot 0.99 + 0.01$
- We initialize all **10 targets** to **0.01** and set the **correct label** (digit) to **0.99**
- Finally, we **train** the network using the **initialized neural network**, the vector of **inputs** and the **target**. The trained network is saved into the file called `trainedNetwork.mat`. We end up with the following output:

```
Command Window
Training in progress....
Time taken: 13.7 s
Saving in progress....
done
fx >>
```



TESTING & EVALUATION

Testing & Evaluation

```
clear;
clc;
error_sum=0;
corr_count=zeros(1,10);
total=zeros(1,10);
percentage=zeros(1,10);
%loading trained model
load("Trained_models\trainedNetwork.mat");
%network_test = neuralNetwork(784,100,10,0.5);
%loading test file
filename= fullfile('datasets','mnist_test.csv');
data = readmatrix(filename);
l=size(data,1);
%testing

for i = 1:l
    inputs = ((data(i,2:end) / 255.0)*0.99+0.01)';
    n=data(i,1);
    total(n+1)=1+total(n+1);
    target = ones(10,1) ./ 100;
    target(n+1,1)=0.99;
    corr=network1.test(inputs,target);

    if corr==1
        corr_count(n+1)=1+corr_count(n+1);
    end
end

for i=1:10
    percentage(i)=(corr_count(i)/total(i))*100;
    fprintf("Digit: %d Accuracy: %.2f%%\n",i-1, percentage(i));
end
%printing accuracy
plot((0:9),percentage);
title('Numbers vs Accuracy');

fprintf("Accuracy of our Neural Network: %.2f%% \n",(sum(corr_count)/l)*100);
disp('End');
```

Testing.m

Explaining the code

Testing:

- The **trained neural network** model is first imported along with the test data from a CSV file (**mnist_test.csv**).
- The **inputs** are **prepared** using the formula: $x_k = v_k / 255.0 \cdot 0.99 + 0.01$
- We initialize all **10 targets** to **0.01** and set the **correct label** (digit) to **0.99**
- The **test** function is then run and the **output (0 or 1)** is stored into the variable **corr**. The sample is **tested** with the network and **corr** is **1** if **correct**.
- If correct, **corr_count** is **incremented** for that **digit**.

Evaluation:

- For **each digit** 0 to 9, the **accuracy (in %)** is calculated and printed
- The **per digit accuracy** is **plotted**
- The **overall accuracy** of the neural network is also printed

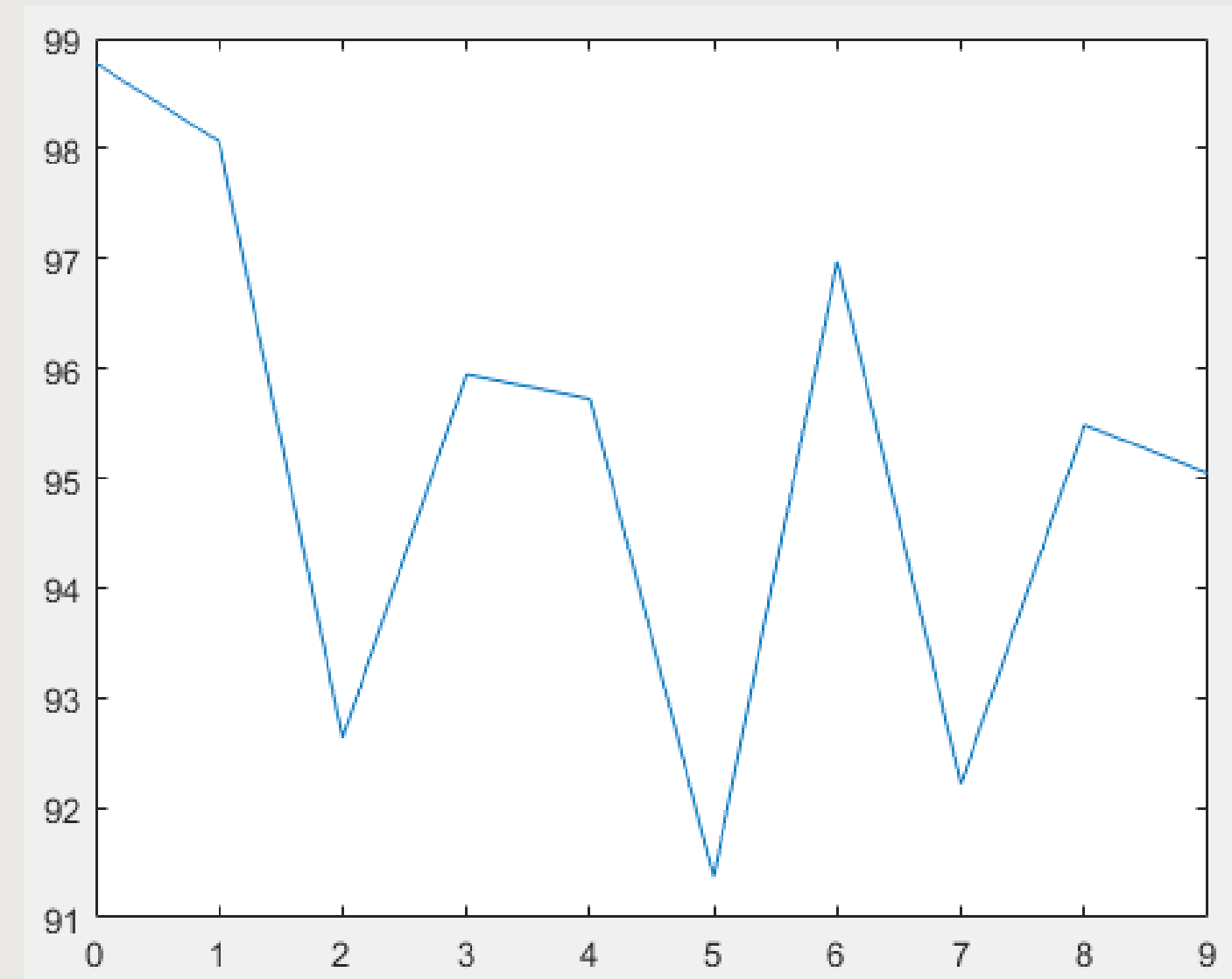


Outputs

Command Window

```
Digit: 0 Accuracy: 98.78  
Digit: 1 Accuracy: 98.06  
Digit: 2 Accuracy: 92.64  
Digit: 3 Accuracy: 95.94  
Digit: 4 Accuracy: 95.72  
Digit: 5 Accuracy: 91.37  
Digit: 6 Accuracy: 96.97  
Digit: 7 Accuracy: 92.22  
Digit: 8 Accuracy: 95.48  
Digit: 9 Accuracy: 95.04  
Accuracy of our Neural Network: 95.27%  
End
```

**Per digit accuracy and average
accuracy on command window**



**Per digit accuracy plotted on a
graph**



FINAL REMARKS

Final Remarks

- The neural network achieved a **test accuracy** of **~95%**
- Noticed accuracy varied with **different random initial weights**, highlighting the sensitivity of neural networks to initialization
- Increasing **hidden** nodes increases **accuracy** and **training time** (and vice versa)
- Instead of initializing weights using the **Xavier** method, if we **initialize all the weights to same numbers** (e.g. 0.5), our neural network guesses the **same number irrespective of inputs** after training.
- Some digits (like **0 and 1**) were classified **more accurately**, while others (like **5 and 7**) showed **more frequent misclassifications**. This could be due to **visual similarity**; eg: 5 can be often confused with 3 and 6; 7 can be confused with 1, etc.
- Future improvements could include using **more training epochs**, **experimenting** with different activation functions or network sizes, and **testing** on our own handwritten digits to further explore the network's capabilities



THANK YOU!

